

Escritura en la API y documentación con OpenAPI

2. Escritura en la API y documentación con OpenAPI

Escritura en la API y documentación con OpenAPI

Ya conoces los DTOs de entrada y de salida, las anotaciones de Bean Validation, `@Valid` y `@Transactional`. Aquí no repites nada de eso: ves esos mismos endpoints de escritura completos, con su código real, y añades algo que todavía no tenías — documentación automática de tu API con OpenAPI.

El LibroController completo (1/2)

Aquí está de nuevo el controller ya explicado, esta vez completo, con los cinco métodos juntos — empezando por los GET que ya conoces:

```
@RestController
@RequestMapping("/api/v1/libros")
@RequiredArgsConstructor
public class LibroController {
    private final LibroService libroService;

    @GetMapping
    public ResponseEntity<List<LibroResponseDTO>> getAll() {
        return ResponseEntity.ok(libroService.findAll());
    }

    @GetMapping("/{id}")
    public ResponseEntity<LibroResponseDTO> getById(@PathVariable Long id) {
        return ResponseEntity.ok(libroService.findById(id));
    }
}
```

El LibroController completo (2/2)

Y los tres métodos de escritura, nuevos en este apartado:

```
@PostMapping
public ResponseEntity<LibroResponseDTO> create(@Valid @RequestBody LibroCreatedDTO dto) {
    return ResponseEntity.status(HttpStatus.CREATED).body(libroService.create(dto));
}

@PutMapping("/{id}")
public ResponseEntity<LibroResponseDTO> update(@PathVariable Long id, @Valid @RequestBody
LibroCreatedDTO dto) {
    return ResponseEntity.ok(libroService.update(id, dto));
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> delete(@PathVariable Long id) {
    libroService.delete(id);
    return ResponseEntity.noContent().build();
}
}
```

Un código distinto según el resultado

getAll y getByld responden con 200 OK. Los tres endpoints de escritura utilizan un código distinto según el resultado que producen:

Verbo	Código habitual de éxito	Qué indica
POST	201 Created	Se ha creado un recurso nuevo.
PUT	200 OK	El recurso se ha reemplazado y se devuelve su estado actualizado.
DELETE	204 No Content	El recurso se ha eliminado y no hay ningún cuerpo que devolver.

Elemento a elemento, lo que hace cada pieza

Elemento	Qué representa
<code>ResponseEntity.status(HttpStatus.CREATED).body(...)</code>	201, con el recurso creado en el cuerpo — la respuesta natural de un POST.
<code>ResponseEntity.ok(...)</code> en el PUT	200, con el recurso ya actualizado en el cuerpo.
<code>ResponseEntity.noContent().build()</code>	204, sin cuerpo — la respuesta natural de un DELETE.
<code>@RequestBody LibroCreateDTO dto</code>	El cuerpo JSON de la petición, convertido automáticamente en un objeto Java.
<code>@Valid</code>	La misma validación que ya conoces: si el DTO incumple alguna restricción, la petición nunca llega a ejecutar el método.

Nada de este código menciona OpenAPI

Fíjate en algo: nada de este código menciona OpenAPI ni Swagger. Eso es justo lo que viene ahora — cómo este mismo controller, sin tocarle una línea, termina documentado y ejecutable desde el navegador.

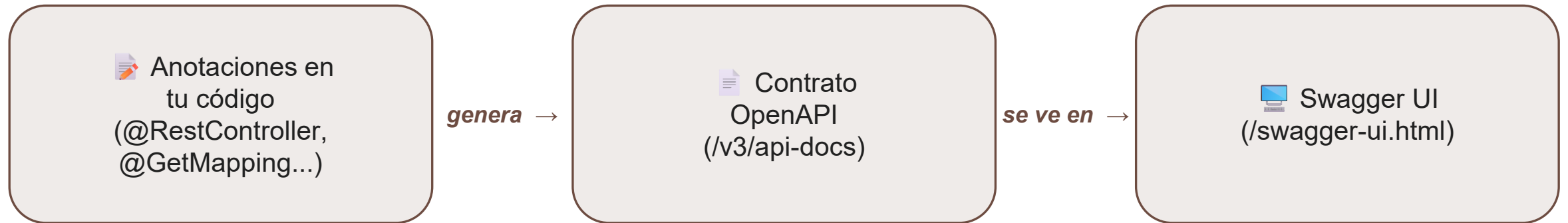
El contrato de una API

Toda esa semántica —qué verbo usar, qué código esperar, si es seguro reintentar— la conoces tú. Pero cuando el consumidor de tu API es otro programa, o un compañero que no ha leído tu código, necesita saber lo mismo sin adivinar.

A esa descripción formal —qué rutas existen, qué verbo usa cada una, qué reciben y qué devuelven— se la llama el contrato de la API.

OpenAPI es el formato estándar más usado para escribir ese contrato (un documento, normalmente en YAML o JSON, que describe rutas, verbos, parámetros y esquemas de datos). Swagger UI es un visor interactivo que lee ese documento y genera, automáticamente, una página web donde se puede explorar la API — y también ejecutarla de verdad.

De la anotación a la página interactiva



Tú no escribes el documento OpenAPI a mano

Lo importante: una librería genera automáticamente gran parte del contrato leyendo las mismas anotaciones (`@RestController`, `@GetMapping`, los DTOs...) que ya utilizas para construir la API.

Así, las rutas, los verbos y los esquemas básicos se mantienen vinculados al código. Más adelante añadirás algunas anotaciones de OpenAPI para completar información que springdoc no puede deducir por sí solo, como la descripción de cada operación.

Esto evita duplicar mucha información

Si añades un nuevo endpoint o cambias los campos de un DTO, la documentación generada refleja esos cambios automáticamente. Algunos detalles, como las descripciones o los códigos de respuesta posibles, sí tendrás que indicarlos mediante anotaciones específicas.

Así se ve Swagger UI, paso a paso

- Abres `/swagger-ui.html` y ves los controllers de tu API agrupados por tag, cada uno desplegable.
- Despliegas, por ejemplo, `POST /api/v1/libros` — Swagger UI te muestra el esquema esperado del cuerpo y un ejemplo de JSON ya relleno.
- Pulsas Try it out: ese ejemplo se vuelve editable. Cambias los valores que quieras.
- Pulsas Execute.

Swagger UI no es solo documentación

Manda peticiones HTTP reales

Al pulsar Execute, Swagger UI construye y envía una petición HTTP real contra tu aplicación. Es equivalente a la que podrías enviar mediante curl o Postman: llega al mismo endpoint y ejecuta el mismo código del controller.

La respuesta que aparece debajo —código de estado, cabeceras y cuerpo— es la respuesta real del servidor. Si ejecutas un POST y se crea un libro, quedará guardado en la base de datos.

Documentando con OpenAPI

La documentación se genera con la dependencia `springdoc-openapi-starter-webmvc-ui` y una clase de configuración mínima:

```
@Configuration
public class OpenApiConfig {

    @Bean
    public OpenAPI libreriaOpenAPI() {
        return new OpenAPI()
            .info(new Info()
                .title("Librería API")
                .version("v1")
                .description("API para gestionar el catálogo de libros, editoriales,
reseñas..."));
    }
}
```

Con solo eso, springdoc hace el resto

Con solo esa dependencia y esa clase, springdoc escanea todos los `@RestController` del proyecto y genera, sin más trabajo por tu parte, la especificación OpenAPI en `/v3/api-docs` y la interfaz visual en `/swagger-ui.html` — los mismos endpoints que el controller ya tenía quedan documentados, y puedes ejecutarlos de verdad con "Try it out".

/v3/api-docs es la especificación en sí

Es el JSON (el 3 es por OpenAPI 3) que describe tus rutas, generado automáticamente por springdoc. Swagger UI no es más que una página que lee ese JSON y lo pinta como interfaz interactiva — no tiene información propia. Por eso las dos rutas van siempre juntas: si Swagger UI no puede llegar a /v3/api-docs, la página carga pero se queda vacía.

Cambiar la ruta de Swagger UI

En tu `application-dev.yml` puedes mover la documentación a otra ruta:

```
springdoc:  
  swagger-ui:  
    path: /documentacion
```

El navegador acaba redirigido a `/swagger-ui/index.html`: esa es la página real. Si más adelante proteges tu API con Spring Security, tendrás que dejar pasar las tres rutas — `/documentacion`, `/swagger-ui/**` y `/v3/api-docs/**` — no solo la que tú has elegido.

operations-sorter: method no hace lo que parece

⚠ No ordena en orden CRUD

Cuidado con lo que promete: no ordena por un criterio "lógico" (GET → POST → PUT → DELETE) — ordena alfabéticamente por el nombre del verbo, así que el resultado real es DELETE, GET, PATCH, POST, PUT. No hay ninguna propiedad simple en springdoc para conseguir el orden CRUD habitual — hay que sobrescribir la función de ordenación de Swagger UI, fuera del alcance de este curso.

Documentando qué puede devolver cada endpoint

Hasta aquí, Swagger UI documenta un endpoint, pero no del todo bien: por defecto solo declara un 200 genérico ("successful operation"), aunque tú ya sabes que `create()` puede devolver un 201, un 400 o un 404. `@ApiResponse`s deja eso explícito.

@ApiResponse en acción

```
@Operation(summary = "Crear un libro nuevo")
@ApiResponses({
    @ApiResponse(responseCode = "201", description = "Libro creado correctamente"),
    @ApiResponse(responseCode = "400", description = "El cuerpo de la petición no supera las validaciones"),
    @ApiResponse(responseCode = "404", description = "La editorial indicada no existe")
})
@PostMapping
public ResponseEntity<LibroResponseDTO> create(@Valid @RequestBody LibroCreateDTO dto) {
    return ResponseEntity.status(HttpStatus.CREATED).body(libroService.create(dto));
}
```

Qué hace cada anotación

`@Operation(summary = ...)` pone un texto legible sobre el endpoint, en vez del nombre técnico del método Java. `@ApiResponse`s agrupa uno o varios `@ApiResponse(responseCode, description)`: cada uno añade, en la sección "Responses" de Swagger UI, una entrada más además del 200 por defecto.

Nada de esto cambia el comportamiento real del endpoint — solo hace que la documentación dependa de ti en vez de generarse a ciegas.

El mismo criterio, en los cinco métodos

Este mismo par de anotaciones se repite en `getById` (200/404), `getAll` (200), `update` (200/400/404) y `delete` (204/404) — no es algo exclusivo de `create()`, es el mismo criterio aplicado a los cinco métodos: qué códigos puede devolver de verdad cada uno.

Ventajas del protocolo estándar, con ejemplos concretos

Para el diagrama de antes —anotaciones, contrato OpenAPI, Swagger UI— existe una condición: tu API tiene que hablar un protocolo que herramientas de terceros ya entiendan de fábrica, sin que tú les enseñes nada.

- Los códigos de estado son universales: cualquier cliente sabe qué significa un 201 o un 404 sin leer tu documentación particular — es parte del estándar HTTP.
- Las herramientas funcionan sin configuración específica: Swagger UI, curl... todas saben "hablar HTTP" de fábrica.

- El contrato de una API describe sus rutas, verbos y datos; OpenAPI es un formato estándar para representarlo. Springdoc genera automáticamente gran parte de ese contrato a partir de los controllers, los DTOs y sus anotaciones.
- Swagger UI agrupa tus endpoints por controller; al desplegar uno ves su esquema, y con "Try it out" + "Execute" mandas una petición HTTP real contra tu aplicación, sin escribir código.
- `@RequestBody` mapea el cuerpo JSON a un objeto Java; `@Valid` activa su validación.

Ideas clave (cont.)

- `@ApiResponses/@ApiResponse` documentan explícitamente qué códigos puede devolver un endpoint; `springdoc.swagger-ui.path` mueve la ruta de entrada, aunque por debajo siempre redirige a `/swagger-ui/index.html`.
- Que Swagger UI y curl puedan hablar los dos con la misma API sin adaptar nada en el servidor es la demostración práctica de qué aporta un protocolo estándar.